

ΠΑΝΕΠΙΣΤΗΜΙΟ ΠΕΙΡΑΙΩΣ
Τμήμα Πληροφορικής



Εργασία Μαθήματος «Αναγνώριση Προτύπων και Μηχανική Μάθηση»

Όνομα φοιτητή – Αρ. Μητρώου	Σμύρης Βασίλειος – ΜΠΣΠ2343
Ημερομηνία παράδοσης	02/09/2024



ΠΙΝΑΚΑΣ ΠΕΡΙΕΧΟΜΕΝΩΝ

1	Εισαγωγή	3
2	Προ-επεξεργασία δεδομένων.....	3
2.1	Φόρτωση δεδομένων	4
2.2	Εύρεση μοναδικών χρηστών και αντικειμένων.....	5
2.3	Περιορισμός συνόλων χρηστών και αντικειμένων.....	5
2.4	Ιστογράμματα συχνότητας αξιολογήσεων χρηστών	6
2.5	Δημιουργία διανυσμάτων προτιμήσεων	8
3	Αλγόριθμοι Ομαδοποίησης Δεδομένων.....	9
3.1	Αλγόριθμος K-Means και μετρικές.....	9
3.2	Ομαδοποίηση χρηστών και γραφική αναπαράσταση αποτελεσμάτων	14
3.3	Σχολιασμός μετρικών	18
4	Αλγόριθμοι Παραγωγής Συστάσεων με Χρήση Τεχνητών Νευρωνικών Δικτύων	20
4.1	Ομαδοποίηση με τη χρήση της απόστασης Jaccard.....	20
4.2	Πρόβλεψη προτιμήσεων χρηστών με τη χρήση των k-πλησιέστερων γειτόνων τους.....	23
5	Βιβλιογραφία	28



1 Εισαγωγή

Η εργασία υλοποιήθηκε με τη χρήση της γλώσσας προγραμματισμού Python. Για την ταξινόμηση των δεδομένων στο δεύτερο ερώτημα, αναπτύχθηκε μια τροποποιημένη έκδοση του αλγορίθμου k-means, η οποία λαμβάνει ως όρισμα μια custom μετρική για τον υπολογισμό της απόστασης ενός χρήστη από τα κέντρα των clusters. Για το δεύτερο ερώτημα, για την ταξινόμηση των χρηστών με την τρίτη μετρική (Jaccard) χρησιμοποιήθηκε ο αλγόριθμος Agglomerative Clustering της βιβλιοθήκης scikit-learn [sklearn, [Agglomerative Clustering](#)], ο οποίος μπορεί να πραγματοποιήσει την ταξινόμηση πάνω στο distance matrix των χρηστών. Για το τρίτο ερώτημα, χρησιμοποιήθηκε η κλάση MLPClassifier της βιβλιοθήκης sci-kit learn, ώστε να υλοποιηθεί ένας Multi-Layer Perceptron ταξινομητής. Οι βιβλιοθήκες που χρησιμοποιήθηκαν για την εργασία είναι οι παρακάτω:

```
import warnings
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from collections import defaultdict
from datetime import datetime
from scipy.sparse import coo_matrix
from scipy.sparse import csr_matrix
from kmeans import KMeansClustering
from sklearn.decomposition import PCA
from sklearn.manifold import TSNE
from sklearn.decomposition import TruncatedSVD
from sklearn.cluster import MiniBatchKMeans, AgglomerativeClustering
from sklearn.metrics import pairwise_distances, mean_absolute_error
from sklearn.model_selection import train_test_split
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense
warnings.filterwarnings("ignore")
```

2 Προ-επεξεργασία δεδομένων

Σε αυτή την ενότητα, παρουσιάζεται ο κώδικας που εισάγει τα δεδομένα στο script και υλοποιεί τα υπο-ερωτήματα του πρώτου ερωτήματος.



2.1 Φόρτωση δεδομένων

Ο παρακάτω κώδικας, αρχικά, φορτώνει τα περιεχόμενα του αρχείου Dataset.npy, το οποίο περιέχει τα δεδομένα για τις αξιολογήσεις των χρηστών. Οι εγγραφές του πίνακα data είναι strings της κάθε γραμμής του αρχείου. Οι εγγραφές χωρίζονται με βάση τα κόμματα και αποθηκεύονται στον δισδιάστατο πίνακα data_list. Από τον πίνακα data_list δημιουργούμε το τελικό pandas dataframe, στις στήλες του οποίου δίνουμε περιγραφικά ονόματα. Επίσης, μετατρέπουμε τις στήλες rating και date στους κατάλληλους τύπους δεδομένων, ώστε να μπορούν να εφαρμοστούν πάνω στα δεδομένα αυτά μαθηματικές πράξεις και συγκρίσεις.

```
# Load the .npy file
data = np.load('Dataset.npy', allow_pickle=True)

# Reshape the data to a list of lists
data_list = [entry.split(',') for entry in data]

# Convert the data to a pandas DataFrame
df = pd.DataFrame(data_list, columns=['user_id', 'movie_id', 'rating',
                                     'date'])

# Convert 'rating' to numeric type and 'date' to datetime type
df['rating'] = pd.to_numeric(df['rating'])
df['date'] = pd.to_datetime(df['date'])

# Print a sample of the data
df.head()
```

Παρακάτω παρουσιάζεται ένα δείγμα από τα δεδομένα, στην τελική μορφή που θα χρησιμοποιηθούν από το υπόλοιπο πρόγραμμα.

	user_id	movie_id	rating	date
0	ur4592644	tt0120884	10	2005-01-16
1	ur3174947	tt0118688	3	2005-01-16
2	ur3780035	tt0387887	8	2005-01-16
3	ur4592628	tt0346491	1	2005-01-16
4	ur3174947	tt0094721	8	2005-01-16



2.2 Εύρεση μοναδικών χρηστών και αντικειμένων

Για την εύρεση του συνόλου των μοναδικών χρηστών και ταινιών στο dataset, καλούμε τη συνάρτηση `nunique` στις στήλες των χρηστών και των ταινιών του dataframe, η οποία επιστρέφει το πλήθος των μοναδικών στοιχείων σε κάθε στήλη.

```
# Display the results
print("Number of unique users ( $|U|$ ):", df['user_id'].nunique())
print("Number of unique items ( $|I|$ ):", df['movie_id'].nunique())
```

Το dataset περιέχει 1499238 μοναδικούς χρήστες και 351109 μοναδικές ταινίες.

Number of unique users ($|U|$): 1499238

Number of unique items ($|I|$): 351109

2.3 Περιορισμός συνόλων χρηστών και αντικειμένων

Σε αυτή την υποενότητα, παρουσιάζεται ο κώδικας για τον περιορισμό των συνόλων των μοναδικών χρηστών και ταινιών, με τη χρήση ενός ελάχιστου (R_{min}) και ενός μέγιστου ορίου (R_{max}) για το πλήθος των αξιολογήσεων που έχουν αφήσει οι χρήστες. Αυτά τα δύο όρια θα πρέπει να αντιπροσωπεύουν ένα χρήσιμο κομμάτι των δεδομένων μας, συμπεριλαμβάνοντας χρήστες που έχουν αφήσει αρκετές κριτικές ώστε να θεωρούνται τακτικοί χρήστες της πλατφόρμας και παραλείποντας χρήστες που δεν παρέχουν έμπιστα δεδομένα συμπεριφοράς. Τέτοιοι χρήστες μπορεί να είναι αυτοί που έχουν αφήσει πολύ λίγες κριτικές, το οποίο μπορεί να υποδεικνύει ότι αυτοί οι χρήστες δε χρησιμοποιούν την πλατφόρμα συχνά ή ότι μπορεί να έχουν δημιουργήσει τον λογαριασμό τους για να στοχοποιήσουν κάποιες συγκεκριμένες ταινίες, για παράδειγμα στα πλαίσια κάποιου *review-bombing*. Άλλη μια κατηγορία χρηστών που μπορεί να μην παρέχουν έμπιστα δεδομένα, είναι αυτοί που έχουν αφήσει πάρα πολλές κριτικές, με αποτέλεσμα να μην παρουσιάζουν κάποιες ιδιαίτερες προτιμήσεις.

Στα πλαίσια της εργασίας, επιλέχθηκε το κομμάτι των χρηστών που έχει αφήσει από 30 έως 35 κριτικές. Αυτό το κομμάτι περιέχει χρήστες με αρκετές κριτικές, ώστε να θεωρηθούν τακτικοί χρήστες. Επίσης, πρέπει να σημειωθεί ότι το κομμάτι των χρηστών είναι τόσο περιορισμένο, λόγω των περιορισμένων πόρων του περιβάλλοντος εκτέλεσης του script. Αρχικά ομαδοποιούμε τις εγγραφές του dataframe με τα δεδομένα των αξιολογήσεων βάσει των χρηστών και εξάγουμε των αριθμό των αξιολογήσεων κάθε χρήστη. Έπειτα, εξάγουμε τους χρήστες από τον πίνακα με τους αριθμούς των αξιολογήσεων, οι οποίοι έχουν αφήσει από R_{min} έως R_{max} αξιολογήσεις και τους αποθηκεύουμε στον πίνακα `filtered_users`. Αυτός ο πίνακας περιέχει τα usernames των χρηστών που πληρούν τις προϋποθέσεις μας. Τέλος, φιλτράρουμε το dataframe χρησιμοποιώντας τα usernames του πίνακα `filtered_users`. Το φιλτραρισμένο dataframe περιέχει μόνο τις εγγραφές για τις κριτικές των χρηστών που έχουν αφήσει από R_{min} έως R_{max} κριτικές. Κρατάμε τους μοναδικούς χρήστες και τις μοναδικές ταινίες του περιορισμένου dataset στις λίστες U_{hat} και I_{hat} και εκτυπώνουμε τα μεγέθη τους.



```
R_min = 30 # Minimum required number of ratings per user
R_max = 35 # Maximum allowed number of ratings per user

# Group by 'user_id' and count the number of ratings per user
user_rating_counts = df.groupby('user_id').size()

# Filter users based on the rating count constraints
filtered_users = user_rating_counts[(user_rating_counts >= R_min) &
(user_rating_counts <= R_max)].index

# Filter the DataFrame to include only rows with the filtered users
filtered_df = df[df['user_id'].isin(filtered_users)]

# Get unique user ids (U_hat) and movie ids (I_hat) from the filtered
DataFrame
U_hat = filtered_df['user_id'].unique()
I_hat = filtered_df['movie_id'].unique()

print("Number of filtered unique users (|U_hat|):", len(U_hat))
print("Number of filtered unique items (|I_hat|):", len(I_hat))
```

Το φιλτραρισμένο dataset για χρήστες που έχουν αφήσει από 30 έως 35 κριτικές, περιέχει 2496 μοναδικούς χρήστες και 31784 μοναδικές ταινίες.

Number of filtered unique users ($|U_{\text{hat}}|$): 2496

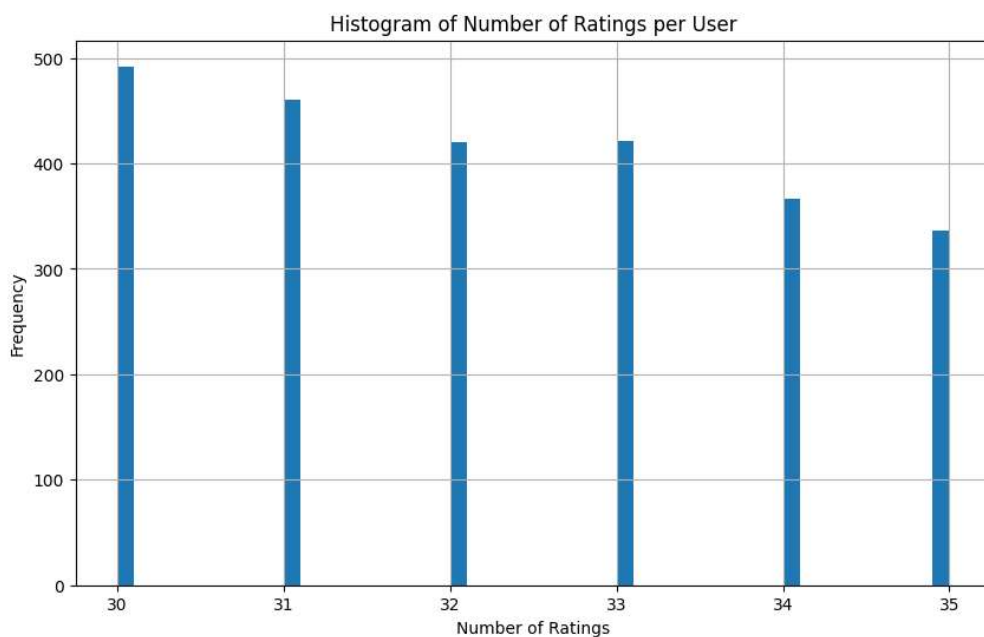
Number of filtered unique items ($|I_{\text{hat}}|$): 31784

2.4 Ιστογράμματα συχνότητας αξιολογήσεων χρηστών

Σε αυτή τη υπο-ενότητα, παρουσιάζεται ο κώδικας που εκτυπώνει τα ιστογράμματα συχνότητας για τις κριτικές, βάσει του πλήθους και του χρονικού εύρους των αξιολογήσεων των χρηστών. Για την εκτύπωση του ιστογράμματος συχνότητας βάσει του πλήθους των αξιολογήσεων κάθε χρήστη, ομαδοποιούμε τα δεδομένα του dataframe μας βάσει των χρηστών και εξάγουμε το πλήθος των αξιολογήσεων κάθε χρήστη.



```
# Plot histogram for the number of ratings per user
plt.figure(figsize=(10, 6))
user_rating_counts_filtered = filtered_df.groupby('user_id').size()
user_rating_counts_filtered.hist(bins=50)
plt.title('Histogram of Number of Ratings per User')
plt.xlabel('Number of Ratings')
plt.ylabel('Frequency')
plt.show()
```



Σχεδιάγραμμα 1 - Ιστόγραμμα συχνότητας πλήθους κριτικών ανα χρήστη.

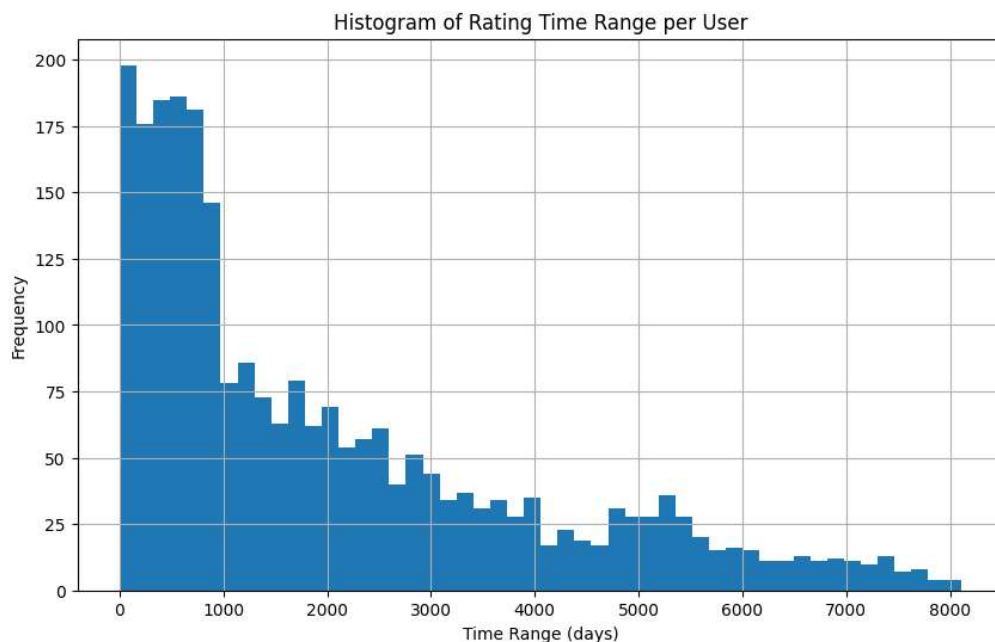
Από το παραπάνω ιστόγραμμα, παρατηρούμε ότι όσο αυξάνεται ο αριθμός των αξιολογήσεων ανά χρήστη, οι χρήστες που έχουν αφήσει αυτόν τον αριθμό αξιολογήσεων τείνουν να μειώνονται. Αυτό δείχνει, ότι οι χρήστες τείνουν να βαθμολογούν ένα περιορισμένο σύνολο ταινιών, πράγμα το οποίο ευθυγραμμίζεται με τα περιορισμένα ενδιαφέροντα σε ήδη ταινιών, ηθοποιούς κλπ. που έχει ένας μέσος χρήστης.

Για την εκτύπωση του ιστογράμματος συχνοτήτων βάσει του χρονικού εύρους των αξιολογήσεων κάθε χρήστη, ομαδοποιούμε τα δεδομένα του dataframe μας βάσει των χρηστών και για κάθε χρήστη υπολογίζουμε τη διαφορά μεταξύ της ελάχιστης ημερομηνίας αξιολογήσεώς του με τη μέγιστη ημερομηνία αξιολόγησης σε μέρες.

```
# Histogram of the time range of ratings per user
plt.figure(figsize=(10, 6))
```



```
rating_time_range = filtered_df.groupby('user_id')['date'].apply(lambda x:
(x.max() - x.min()).days)
rating_time_range.hist(bins=50)
plt.title('Histogram of Rating Time Range per User')
plt.xlabel('Time Range (days)')
plt.ylabel('Frequency')
plt.show()
```



Σχεδιάγραμμα 2 - Ιστόγραμμα συχνότητας χρονικού εύρους κριτικών ανά χρήστη.

Από το παραπάνω ιστόγραμμα, διακρίνουμε ότι όσο αυξάνεται το χρονικό εύρος των κριτικών, ο αριθμός των χρηστών που έχουν αφήσει κριτικές στο χρονικό εύρος αυτό τείνει να μειώνεται, το οποίο υποδεικνύει ότι οι χρήστες τείνουν να είναι ενεργοί στην πλατφόρμα για ένα περιορισμένο χρονικό διάστημα.

2.5 Δημιουργία διανυσμάτων προτιμήσεων

Για την κατασκευή των διανυσμάτων προτιμήσεων των χρηστών, αρχικά δημιουργούμε mappings των id των χρηστών και των ταινιών σε αριθμητικά ids. Αυτά τα αριθμητικά ids θα διευκολύνουν αργότερα την προσπέλαση των δεδομένων μας από τους αλγορίθμους ταξινόμησης και τα νευρωνικά δίκτυα. Στο dataframe με τις αξιολογήσεις, αντικαθιστούμε τα αλφαριθμητικά ids με τα αριθμητικά sequential ids και δημιουργούμε μια sparse matrix με τα διανύσματα προτιμήσεων των χρηστών, όπου κάθε στήλη αντιπροσωπεύει έναν χρήστη και κάθε γραμμή αντιπροσωπεύει τη βαθμολογία του κάθε χρήστη για μια ταινία. Επιλέγουμε sparse matrix λόγω του πολύ μεγάλου μεγέθους των δεδομένων που προκύπτουν από τη



δημιουργία των διανυσμάτων προτιμήσεων, καθώς, σε αντίθεση με τα αρχικά δεδομένα μας, τα οποία περιέχουν εγγραφές μόνο για τις ταινίες που έχει βαθμολογήσει κάθε χρήστης, τα διανύσματα προτιμήσεων πρέπει να περιέχουν στοιχεία και για τις ταινίες που δεν έχει βαθμολογήσει ένας χρήστης, ώστε όλα τα διανύσματα να έχουν το ίδιο μέγεθος και το κάθε στοιχείο να αφορά την ίδια ταινία με τα αντίστοιχα στοιχεία των άλλων διανυσμάτων. Η δομή `csr matrix` της βιβλιοθήκης `scipy`, συμπιέζει αυτά τα πολύ αραιά δεδομένα, καθώς περιέχει εγγραφές με τις συντεταγμένες των μη-μηδενικών στοιχείων και των τιμών τους.

```
# Create mappings for user_id and movie_id to integer indices
user_mapping = {user_id: idx for idx, user_id in enumerate(U_hat)}
movie_mapping = {movie_id: idx for idx, movie_id in enumerate(I_hat)}

# Map user_id and movie_id to integer indices in the filtered dataframe
filtered_df['user_idx'] = filtered_df['user_id'].map(user_mapping)
filtered_df['movie_idx'] = filtered_df['movie_id'].map(movie_mapping)

# Create the sparse user-item matrix
user_item_matrix = coo_matrix((filtered_df['rating'],
(filtered_df['user_idx'], filtered_df['movie_idx'])), shape=(len(U_hat),
len(I_hat))).tocsr()

# Display some information about the sparse matrix
print(f"Sparse matrix shape: {user_item_matrix.shape}")
print(f"Number of non-zero entries: {user_item_matrix.nnz}")
print("Example for user 0: \n", user_item_matrix[0])
```

3 Αλγόριθμοι Ομαδοποίησης Δεδομένων

Σε αυτή την ενότητα, ομαδοποιούμε τους χρήστες βάσει των προτιμήσεών τους, με τη χρήση του αλγορίθμου `k-means` και τις τροποποιημένες συναρτήσεις ευκλείδειας και συνημιτονικής απόστασης.

3.1 Αλγόριθμος K-Means και μετρικές

Για τις ανάγκες του ερωτήματος, καθώς η υλοποίηση του αλγορίθμου `k-means` που παρέχει το `scikit-learn` δεν υποστηρίζει εκτέλεση του αλγορίθμου πάνω σε πίνακες αποστάσεων ή τον ορισμό κάποιας custom μετρικής αποστάσεων[`sklearn`, [KMeans](#)] δημιουργήθηκε η κλάση `KMeansClustering`, η οποία υλοποιεί τον αλγόριθμο `K-Means`[`Pik12`, p.658--662] και δίνει τη δυνατότητα επιλογής μιας μετρικής από αυτές του ερωτήματος για τον υπολογισμό των αποστάσεων των χρηστών από τα κέντρα των `clusters`. Στο παρακάτω snippet, φαίνεται ο constructor της κλάσης. Ο constructor λαμβάνει 3 ορίσματα, το `k`, τον



αριθμό των συστάδων στις οποίες θα ταξινομήσει ο αλγόριθμος τα δεδομένα, το `distance_metric`, την μετρική που θα χρησιμοποιήσει ο αλγόριθμος για τις αποστάσεις, και το `random_state`, το οποίο επηρεάζει την αρχική τυχαία επιλογή των κεντροειδών των συστάδων.

```
class KMeansClustering:
    def __init__(self, k=3, distance_metric='euclidean', random_state=None):
        self.k = k
        self.centroids = None
        self.distance_metric = distance_metric
        self.distance_function = self._get_distance_function(distance_metric)
        self.random_state = random_state
        self.rng = np.random.default_rng(random_state)
```

Οι συναρτήσεις των αποστάσεων έχουν δηλωθεί μέσα στην κλάση `KMeansClustering`. Παρακάτω παρουσιάζεται ο κώδικας της ευκλείδειας (`euclidean`) απόστασης. Η συνάρτηση λαμβάνει ως παραμέτρους δύο διανύσματα προτιμήσεων προς σύγκριση ως *sparse matrices* και τα μετατρέπει σε πυκνές μονοδιάστατες λίστες. Έπειτα, υπολογίζονται οι μάσκες των διανυσμάτων, δυαδικά διανύσματα όπου οι κάθε τιμή είναι 0 αν ο χρήστης δεν έχει βαθμολογήσει μια ταινία και 1 αν ο χρήστης την έχει βαθμολογήσει. Πριν συγκριθούν τα δύο διανύσματα προτιμήσεων, ελέγχεται αν τα διανύσματα μπορούν να συγκριθούν. Για να μπορούν να συγκριθούν δύο διανύσματα, πρέπει οι δύο χρήστες να έχουν βαθμολογήσει τουλάχιστον μία κοινή ταινία. Οι μάσκες πολλαπλασιάζονται και υπολογίζεται το άθροισμα των στοιχείων της μάσκας που προκύπτει. Αν είναι μηδέν, σημαίνει ότι οι χρήστες δεν έχουν βαθμολογήσει καμία κοινή ταινία και επιστρέφεται ο μεγαλύτερος δυνατός ακέραιος ως η διαφορά τους. Η μηδενική βαθμολογία σημαίνει ότι ο χρήστης δεν έχει βαθμολογήσει μια ταινία και όχι ότι την έχει βαθμολογήσει με μηδέν, το οποίο σημαίνει ότι μια σύγκριση για μια ταινία που έχει βαθμολογηθεί μόνο από τον ένα χρήστη, και κατ' επέκταση μια σύγκριση μεταξύ δύο χρηστών που δεν έχουν βαθμολογήσει κοινές ταινίες, δεν είναι έγκυρες. Παρ' όλα αυτά, σαν συμβιβασμό, θεωρούμε ότι δυο χρήστες που δεν έχουν βαθμολογήσει κοινές ταινίες έχουν άπειρη απόσταση μεταξύ τους, ώστε να μειώσουμε τις πιθανότητες αυτοί οι χρήστες να ταξινομηθούν στα ίδια clusters. Εάν οι τα διανύσματα κριθούν συγκρίσιμα, εφαρμόζεται ο τροποποιημένος τύπος της ευκλείδειας απόστασης που δεν λαμβάνει υπ' όψη τις ταινίες για τις οποίες κάποιος από τους δύο χρήστες δεν έχει αφήσει βαθμολογία.

$$dist_{euclidean}(R_u, R_v) = \sqrt{\sum_{k=1}^m |R_u(k) - R_v(k)|^2 \lambda_u(k) \lambda_v(k)}$$

$$\lambda_j(k) = \begin{cases} 1, & R(u_j, i_k) > 0; \\ 0, & R(u_j, i_k) = 0. \end{cases}$$

Εικόνα 1 - Τύπος ευκλείδειας απόστασης με χρήστη συνάρτησης φιλτραρίσματος μη κοινών στοιχείων.



```
@staticmethod
# Function to calculate the euclidean distance between two user ranking
vectors
def euclidean_dist(R_u, R_v):
    # Extract required vectors from the sparse data and calculate lambdas
    R_u = R_u.toarray().flatten() if isinstance(R_u, csr_matrix) else
R_u.flatten()
    R_v = R_v.toarray().flatten() if isinstance(R_v, csr_matrix) else
R_v.flatten()
    lambda_u = np.where(R_u > 0, 1, 0)
    lambda_v = np.where(R_v > 0, 1, 0)

    # Check if vectors are valid for comparison
    mask = lambda_u * lambda_v
    if np.sum(mask) == 0:
        return float('inf')

    # Calculate distance with the euclidean formula
    dist = np.sqrt(np.sum(np.square((R_u - R_v) * mask)))
    return dist
```

Ο παρακάτω κώδικας υλοποιεί τον υπολογισμό της συνημιτονικής απόστασης. Όπως και στη συνάρτηση της ευκλείδειας απόστασης, λαμβάνονται ως ορίσματα οι sparse matrices δύο διανυσμάτων προτιμήσεων, μετατρέπονται σε dense format, υπολογίζονται οι μάσκες τους και ελέγχεται εάν τα δύο διανύσματα είναι συγκρίσιμα. Εάν, τα διανύσματα, είναι συγκρίσιμα, υπολογίζονται τα components του κλάσματος της συνημιτονικής ομοιότητας και υπολογίζεται η απόσταση. Σε περίπτωση που κάποιο μέρος του παρονομαστή είναι μηδενικό, επιστρέφεται άπειρη απόσταση.

$$dist_{cosine}(R_u, R_v) = 1 - \left| \frac{\sum_{k=1}^m R_u(k) R_v(k) \lambda_u(k) \lambda_v(k)}{\sqrt{\sum_{k=1}^m R_u(k)^2 \lambda_u(k) \lambda_v(k)} \sqrt{\sum_{k=1}^m R_v(k)^2 \lambda_u(k) \lambda_v(k)}} \right|$$

Εικόνα 2 - Τύπος συνημιτονικής απόστασης χρήστη συνάρτησης φιλτραρίσματος μη κοινών στοιχείων.

```
@staticmethod
# Function to calculate the cosine distance between two user ranking
vectors
def cosine_dist(R_u, R_v):
    # Extract required vectors from the sparse data and calculate lambdas
    R_u = R_u.toarray().flatten() if isinstance(R_u, csr_matrix) else
R_u.flatten()
```



```
R_v = R_v.toarray().flatten() if isinstance(R_v, csr_matrix) else
R_v.flatten()
lambda_u = np.where(R_u > 0, 1, 0)
lambda_v = np.where(R_v > 0, 1, 0)

# Check if vectors are valid for comparison
mask = lambda_u * lambda_v
if np.sum(mask) == 0:
    return float('inf')

# Calculate similarity fraction components
numerator = np.sum(R_u * R_v * mask)

denominator_u = np.sqrt(np.sum(np.square(R_u) * mask))
denominator_v = np.sqrt(np.sum(np.square(R_v) * mask))

if denominator_u == 0 or denominator_v == 0:
    return float('inf')

# Calculate distance using the cosine formula
dist = 1 - np.abs(numerator / (denominator_u * denominator_v))

return dist
```

Η συνάρτηση fit πραγματοποιεί την εκτέλεση του αλγορίθμου k-means. Χρησιμοποιεί τον αριθμό των συστάδων και τη μετρική που λαμβάνει ο constructor της κλάσης. Σαν παραμέτρους, λαμβάνει το dataset προς ταξινόμηση, η παράμετρος X, και τον αριθμό των μέγιστων επαναλήψεων για τον αλγόριθμο, η παράμετρος max_iterations. Για να λειτουργήσει σωστά ο αλγόριθμος και οι μετρικές, το dataset εισόδου πρέπει να είναι σε sparse matrix format. Ο έλεγχος αυτός γίνεται στην αρχή της συνάρτησης. Αρχικά, επιλέγονται τυχαία k κεντροειδή από το dataset.

```
def fit(self, X, max_iterations=100):
    if not isspmatrix_csr(X):
        raise ValueError("X should be a CSR matrix")

    # Initialize centroids with random samples from the data
    n_samples, n_features = X.shape
    random_indices = self.rng.choice(n_samples, self.k, replace=False)
    self.centroids = X[random_indices].toarray() # Initial centroids as
dense array for computation
```



Σε κάθε επανάληψη, αρχικοποιείται με μηδενικά η λίστα labels, στην οποία θα αποθηκεύονται τα cluster labels των δειγμάτων. Για κάθε δείγμα, υπολογίζεται η απόστασή του από τα κεντροειδή των clusters και επιλέγεται ως label του το cluster του οποίου το κεντροειδές είναι πιο κοντά στο δείγμα.

```
for iterations in range(max_iterations):
    labels = np.zeros(n_samples, dtype=int)
    # Compute sample distances from the centroids
    for i in range(n_samples):
        data_point = X.getrow(i).toarray()
        distances = [self.distance_function(data_point, centroid) for
centroid in self.centroids]
        labels[i] = np.argmin(distances)
```

Τα κεντροειδή αρχικοποιούνται ως μηδενικά διανύσματα και δημιουργείται ο πίνακας counts, ο οποίος θα κρατά τα πλήθη των δειγμάτων της κάθε συστάδας. Για κάθε δείγμα, ανακτούμε το cluster του και στο κεντροειδές του cluster προσθέτουμε τις τιμές των features του δείγματος. Επίσης, αυξάνουμε το πλήθος των στοιχείων του cluster κατά ένα. Αυτό το loop μας δίνει σαν αποτέλεσμα τα κεντροειδή των cluster ως τα αθροίσματα των διανυσμάτων των cluster (cluster_centers) και τα πλήθη των στοιχείων που ανήκουν σε κάθε cluster (counts).

```
cluster_centers = np.zeros((self.k, n_features))
counts = np.zeros(self.k)

for i in range(n_samples):
    cluster_num = labels[i]
    cluster_centers[cluster_num] += X.getrow(i).toarray().ravel()
    counts[cluster_num] += 1
```

Για κάθε cluster, εάν το cluster είναι άδειο, τότε επιλέγουμε ως κεντροειδές του ένα τυχαίο δείγμα από το dataset, αλλιώς έχουμε για αυτό το κεντροειδές του ως το άθροισμα των διανυσμάτων των στοιχείων του, το οποίο διαιρούμε με το πλήθος των στοιχείων του και παίρνουμε το κεντροειδές του cluster για την επόμενη επανάληψη. Τέλος, ελέγχουμε κατά πόσο τα νέα κεντροειδή διαφέρουν από τα κεντροειδή της προηγούμενης επανάληψης. Εάν η διαφορά είναι μικρότερη από 0,0001, τότε ο αλγόριθμος τελειώνει την εκτέλεσή του πριν ολοκληρωθεί ο προκαθορισμένος αριθμός επαναλήψεων. Η συνάρτηση επιστρέφει τα labels των δειγμάτων.

```
for i in range(self.k):
    if counts[i] == 0:
        # Reinitialize empty clusters
```



```
        cluster_centers[i] =  
X[np.random.choice(n_samples)].toarray()  
    else:  
        cluster_centers[i] /= counts[i]  
  
    if np.linalg.norm(self.centroids - cluster_centers) < 0.0001:  
        break  
    else:  
        self.centroids = cluster_centers  
return labels
```

3.2 Ομαδοποίηση χρηστών και γραφική αναπαράσταση αποτελεσμάτων

Η παρακάτω συνάρτηση θα χρησιμοποιηθεί κατά τη διαδικασία ταξινόμησης για την οπτικοποίηση των αποτελεσμάτων για κάθε αριθμό συστάδων και μετρική

```
# Function to plot clusters  
def plot_clusters(reduced_data, labels, title):  
    plt.figure(figsize=(8, 6))  
    scatter = plt.scatter(reduced_data[:, 0], reduced_data[:, 1], c=labels,  
cmap='tab20', marker='o')  
    plt.title(title)  
    plt.xlabel('Component 1')  
    plt.ylabel('Component 2')  
    plt.colorbar(scatter)  
    plt.show()
```

Η συσταδοποίηση πραγματοποιείται για 10, 15 και 20 συστάδες, και για τις δύο μετρικές του ερωτήματος. Για κάθε αριθμό συστάδων και μετρική, εκτελείται συσταδοποίηση με K-Means με τη χρήση της κλάσης της προηγούμενης υπο-ενότητας. Για την οπτικοποίηση των αποτελεσμάτων της κάθε εκτέλεσης, τα οι διαστάσεις των δεδομένων πρώτα μειώνονται σε 50 με τη μέθοδο SVD[Pik12, p.353-361] και έπειτα σε 2 με t-SNE.

```
L_values = [10, 15, 20] # Different values of L (number of clusters)  
metrics = ['euclidean', 'cosine']  
  
# Loop through each metric and each value of L  
for L in L_values:  
    for metric in metrics:  
        kmeans = KMeansClustering(k=L, distance_metric=metric,  
random_state=0)
```

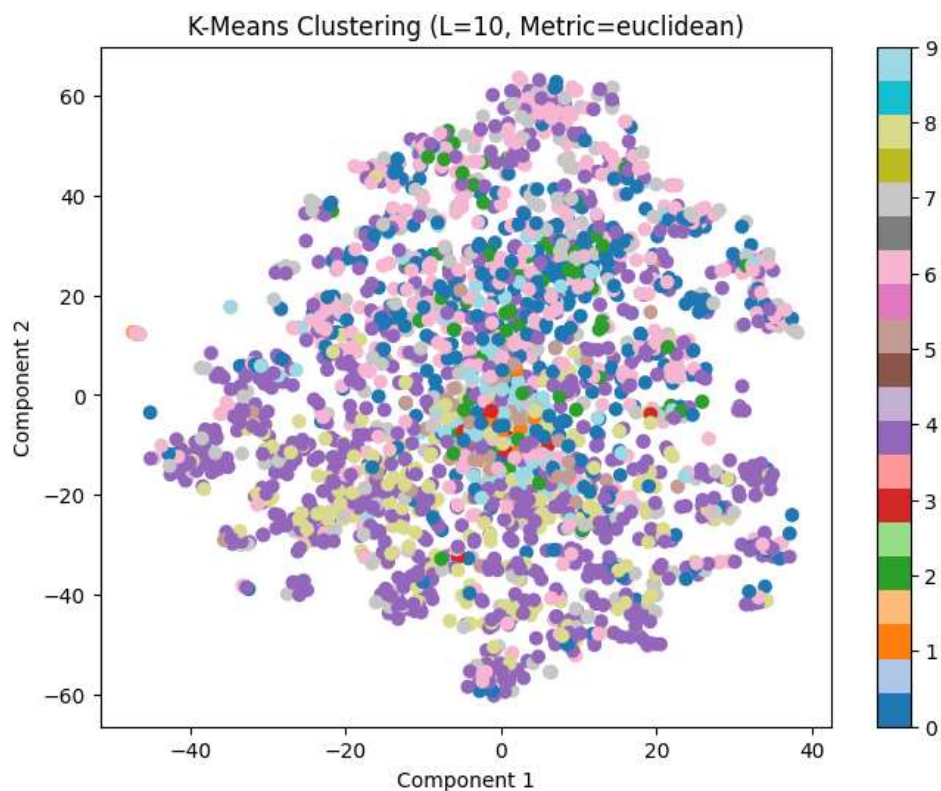


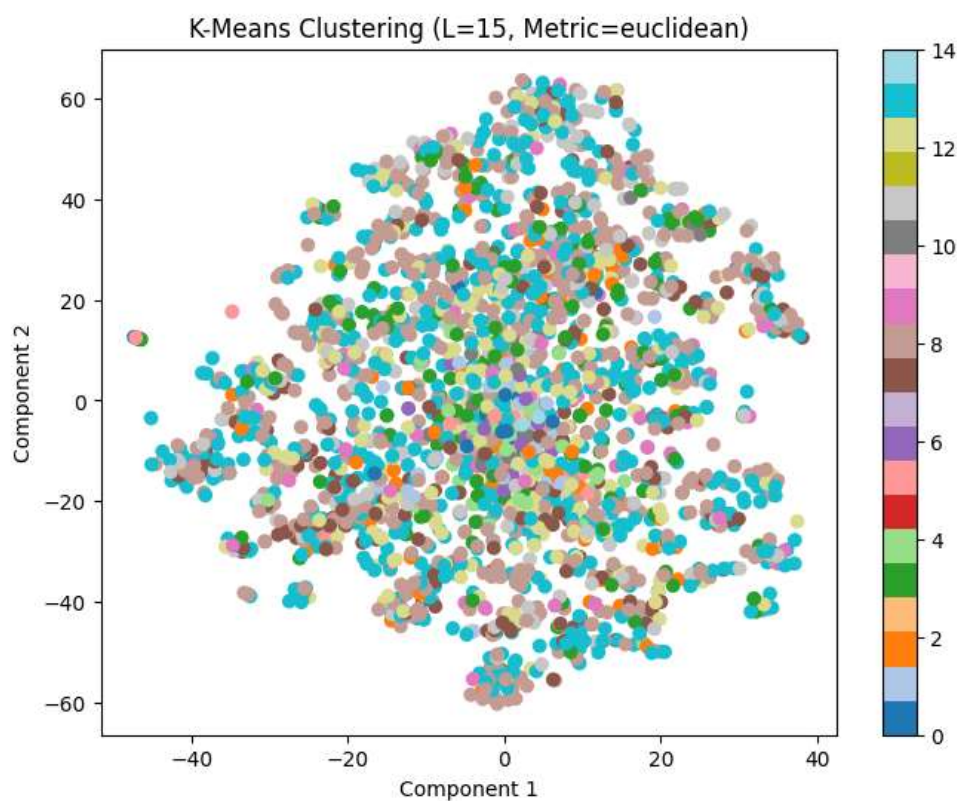
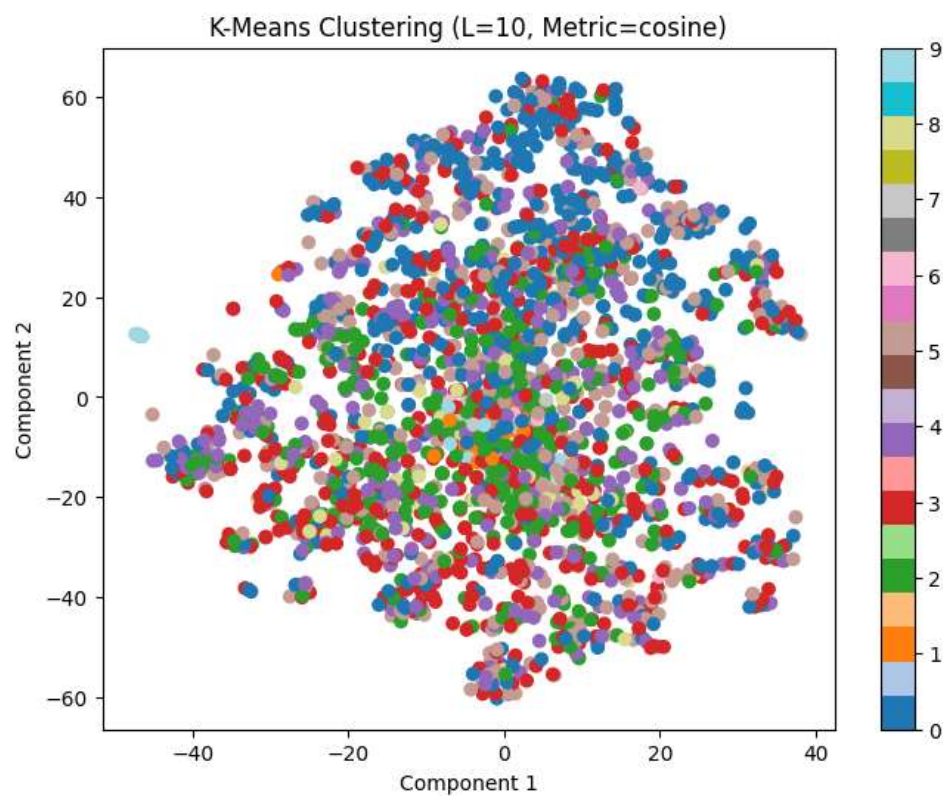
```
# Fit the model on the data
labels = kmeans.fit(user_item_matrix)

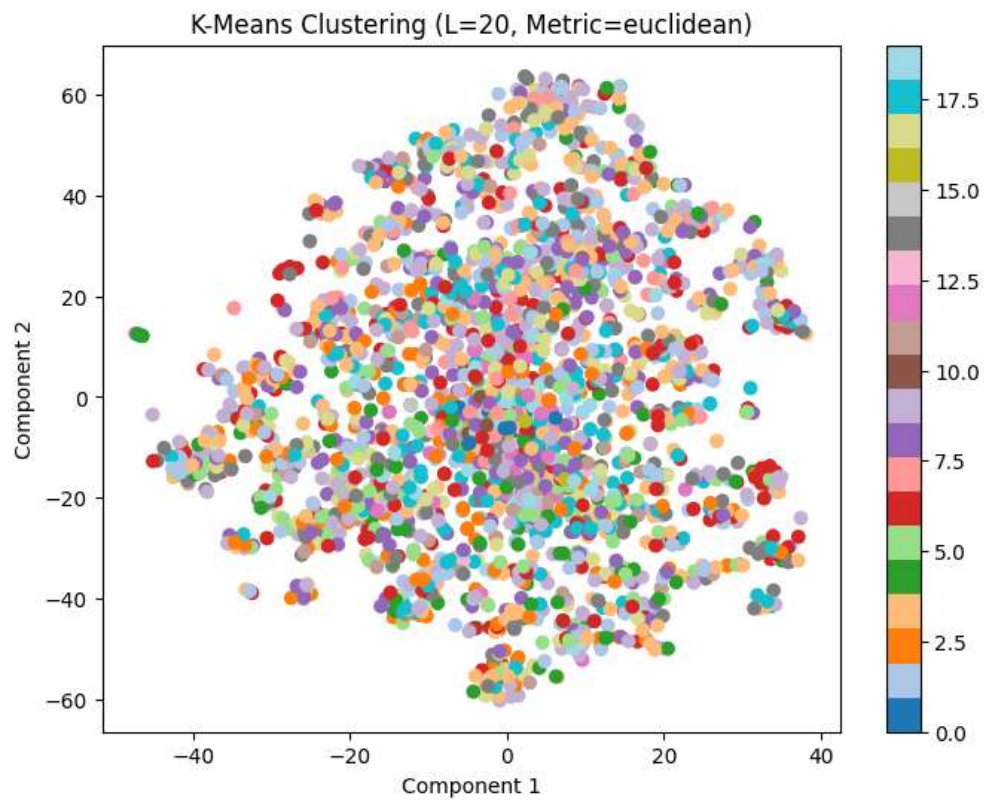
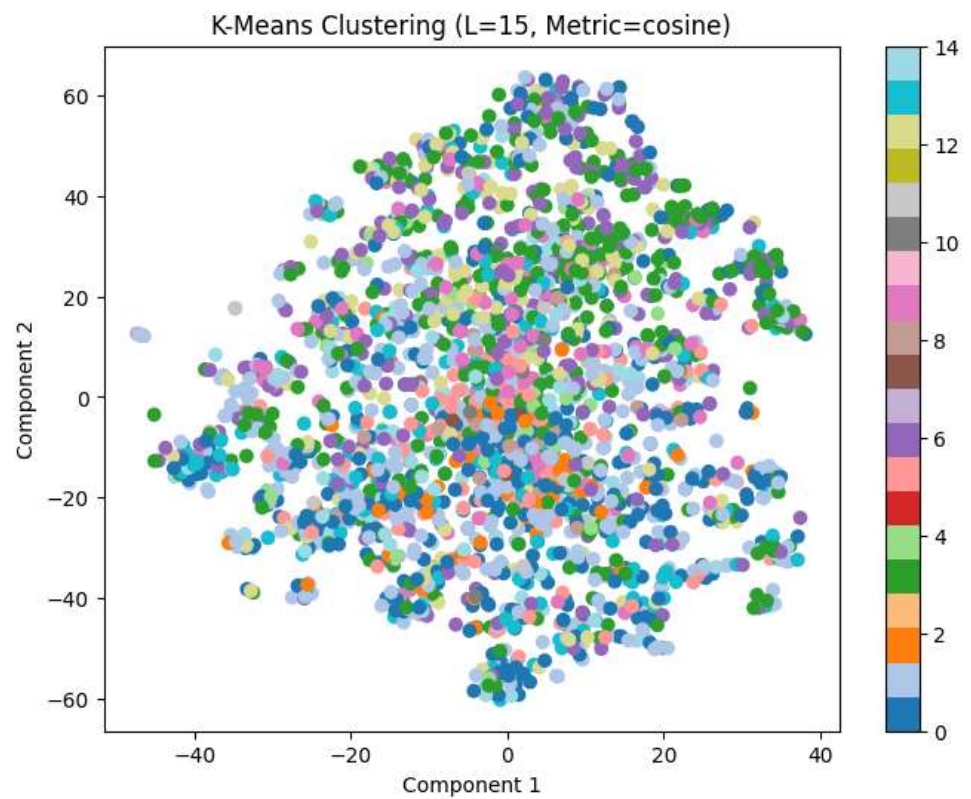
# Reduce dimensions of the data
svd = TruncatedSVD(n_components=50, random_state=42)
svd_data = svd.fit_transform(user_item_matrix)
reducer = TSNE(n_components=2, random_state=42)
reduced_data = reducer.fit_transform(svd_data)

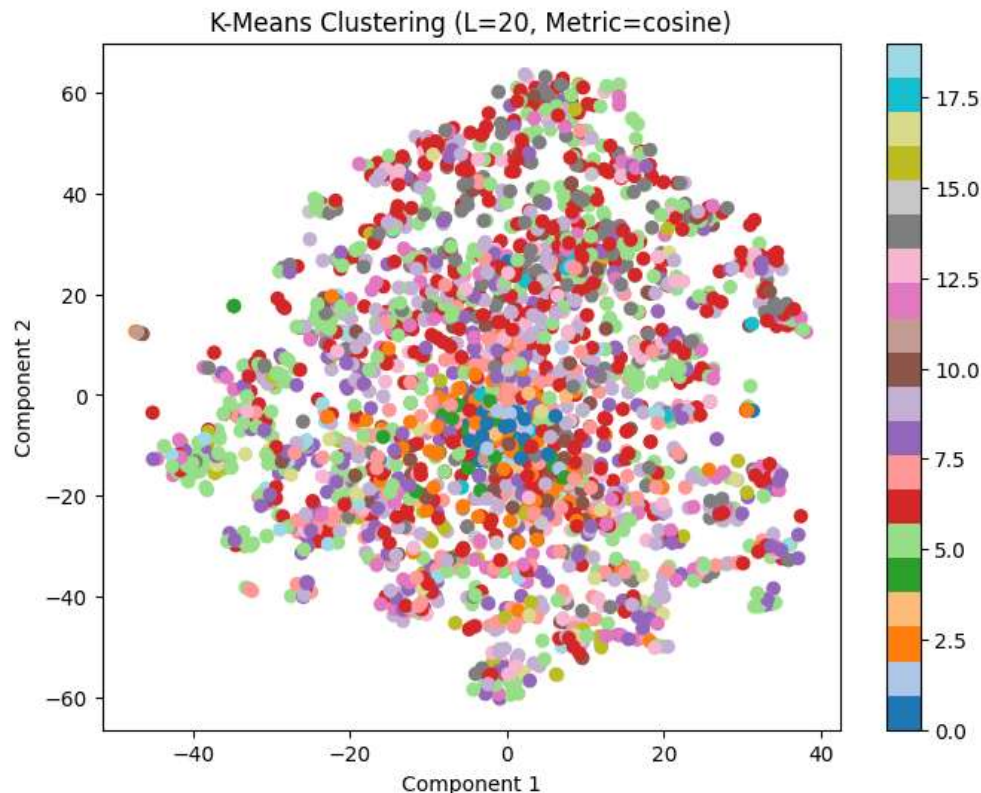
# Plot the resulting clusters
title = f'K-Means Clustering (L={L}, Metric={metric})'
plot_clusters(reduced_data, labels, title)
```

Στις παρακάτω εικόνες παρουσιάζονται οι οπτικοποιήσεις των αποτελεσμάτων.









3.3 Σχολιασμός μετρικών

Και οι δύο μετρικές του ερωτήματος είναι καλύτερες στη συσταδοποίηση των δεδομένων από την απλή ευκλείδεια απόσταση του k-means, καθώς εάν ένας χρήστης έχει βαθμολογήσει μια ταινία, ενώ ένας άλλος όχι, δεν θα πρέπει να υπολογίσουμε τη διαφορά μεταξύ των κριτικών τους, καθώς αυτή η σύγκριση δεν θα ήταν έγκυρη. Για παράδειγμα, αν ένας χρήστης έχει βαθμολογήσει μια ταινία με 7 και ένας άλλος χρήστης δεν έχει βαθμολογήσει την ίδια ταινία, που σημαίνει ότι η κριτική του δεύτερου χρήστη έχει την αρχική τιμή 0, η σπάνια ευκλείδεια απόσταση που χρησιμοποιεί ο k-means θα προσέθετε την διαφορά 7 των δύο κριτικών στην διαφορά των χρηστών, το οποίο δεν είναι έγκυρο, καθώς 0 σημαίνει ότι ο χρήστης δεν έχει εκφράσει γνώμη για εκείνη την ταινία και όχι ότι την έχει βαθμολογήσει με 0, επομένως η διαφορά μεταξύ των δύο κριτικών δε μπορεί να υπολογιστεί. Οι μετρικές του ερωτήματος, λαμβάνουν υπ' όψη το προηγούμενο και δεν προσθέτουν στην απόσταση μεταξύ δύο χρηστών τις διαφορές των ταινιών τις οποίες ένας από τους δύο χρήστες δεν έχει βαθμολογήσει. Συνοπτικά, οι μετρικές του ερωτήματος μετρούν την απόσταση μεταξύ δύο χρηστών ως το άθροισμα των διαφορών των ταινιών που έχουν βαθμολογήσει και οι δύο χρήστες. Ένα πρόβλημα των μετρικών είναι ότι εάν δύο χρήστες δεν έχουν βαθμολογήσει καμία κοινή ταινία, η διαφορά τους θα είναι μηδέν, το οποίο σημαίνει ότι οι χρήστες είναι παρόμοιοι, πράγμα ψευδές, καθώς οι σύγκριση μεταξύ δύο χρηστών που δεν έχουν βαθμολογήσει κοινές ταινίες είναι μη έγκυρη. Στις συναρτήσεις αποστάσεων που έχουν υλοποιηθεί στο πρόγραμμα, αυτή η περίπτωση αντιμετωπίζεται με την επιστροφή ενός πολύ μεγάλου αριθμού. Αυτός ο πολύ μεγάλος αριθμός συνεχίζει να μην είναι έγκυρος, καθώς



δεν μπορεί να υπολογιστεί απόσταση μεταξύ χρηστών που δεν έχουν βαθμολογήσει κοινές ταινίες, αλλά διαβεβαιώνει ότι θα είναι λιγότερο πιθανό δύο τέτοιοι χρήστες να θεωρηθούν όμοιοι. Ένα ακόμα πρόβλημα με τις μετρικές, είναι ότι προσθέτουν υπολογιστικό overhead, καθώς πρέπει να υπολογίζονται κάθε φορά οι μάσκες των διανυσμάτων προτιμήσεων.

Συγκριτικά μεταξύ τους, οι δύο μετρικές δίνουν διαφορετικές πληροφορίες για τα δεδομένα. Η ευκλείδεια μετρική είναι καλύτερη στο να συσταδοποιεί μαζί χρήστες που βαθμολογούν ταινίες με τον ίδιο τρόπο. Θεωρούν παρόμοιους τους χρήστες που δίνουν βαθμολογίες κοντινές μεταξύ τους, που σημαίνει ότι χρήστες που βαθμολογούν τις ίδιες ταινίες πολύ υψηλά ή πολύ χαμηλά θα τοποθετηθούν στην ίδια συστάδα. Η συνημιτονική απόσταση ομαδοποιεί μαζί χρήστες που βαθμολογούν τις ίδιες ταινίες υψηλά ή χαμηλά, αλλά όχι βάσει της απόλυτης τιμής των βαθμολογιών, αλλά βάσει του μοτίβου βαθμολόγησης που παρουσιάζεται κατά μήκος ολόκληρων των διανυσμάτων προτιμήσεων. Αυτό σημαίνει ότι πάλι χρήστες που δίνουν παρόμοιες βαθμολογίες στις ίδιες ταινίες θα ομαδοποιηθούν μαζί, καθώς τις βαθμολογούν με παρόμοιο μοτίβο, αλλά θα ομαδοποιηθούν μαζί και χρήστες που δίνουν ανάλογες βαθμολογίες στις ίδιες ταινίες. Για παράδειγμα, δύο χρήστες που δίνουν στις ίδιες ταινίες ομοίως χαμηλότερες ή υψηλότερες βαθμολογίες, οι οποίες είναι με παρόμοιο τρόπο μεγαλύτερες ή μικρότερες από βαθμολογίες άλλων ταινιών, θα ομαδοποιηθούν μαζί, όπως για παράδειγμα ένας χρήστης που έχει βαθμολογήσει δυο ταινίες με 2 και 4 κι ένας χρήστης που έχει βαθμολογήσει δυο ταινίες με 4 και 8. Και στις δυο περιπτώσεις η βαθμολογία της δεύτερης ταινίας είναι μεγαλύτερη και διπλάσια από αυτή της πρώτης, δηλαδή οι τιμές κατά μήκος του διανύσματος αλλάζουν με τον ίδιο τρόπο. Η ευκλείδεια απόσταση ομαδοποιεί μαζί χρήστες που βαθμολογούν τις ίδιες ταινίες παρομοίως, αλλά παρ' όλο που είναι χρήσιμη αυτή η μετρική, η συνημιτονική απόσταση μπορεί να εντοπίσει και κάποιες παραπάνω πληροφορίες για τη συμπεριφορά του χρήστη, όπως για παράδειγμα την προτίμηση σε κάποια συγκεκριμένα είδη ταινιών, εάν έχει βαθμολογήσει ταινίες ενός είδους περισσότερο από ταινίες άλλων ειδών.



4 Αλγόριθμοι Παραγωγής Συστάσεων με Χρήση Τεχνητών Νευρωνικών Δικτύων

Σε αυτή την ενότητα, παρουσιάζεται η ομαδοποίηση των δεδομένων με τη χρήση της απόστασης Jaccard, εκπαίδευση νευρωνικού δικτύου με τη χρήση της μετρικής αυτής και οι πρόβλεψη των προτιμήσεων ενός χρήστη βάσει της προηγούμενης εκπαίδευσης.

4.1 Ομαδοποίηση με τη χρήση της απόστασης Jaccard

Σε αυτή την ενότητα, παρουσιάζεται ο κώδικας που πραγματοποιεί την ομαδοποίηση των χρηστών με τη μετρική απόστασης του ερωτήματος, γνωστή ως απόσταση Jaccard, και αναφέρονται τα μειονεκτήματά της συγκριτικά με τις μετρικές της προηγούμενης ενότητας. Η μετρική Jaccard μετρά την απόσταση δύο διανυσμάτων ως 1 μείον το λόγο του πλήθους των στοιχείων της τομής των συνόλων των στοιχείων τους προς το πλήθος των στοιχείων της ένωσης των συνόλων των στοιχείων τους. Για να λειτουργήσει σωστά η μετρική Jaccard, πρώτα δημιουργούμε μια δυαδική μήτρα, με κάθε στήλη να αντιπροσωπεύει ένα χρήστη και κάθε γραμμή να αντιπροσωπεύει τις βαθμολογίες των χρηστών για μια ταινία. Ένα στοιχείο με τιμή 1 σημαίνει ότι ο χρήστης έχει αφήσει βαθμολογία για τη συγκεκριμένη ταινία, ενώ ένα στοιχείο με τιμή 0 σημαίνει ότι δεν έχει. Για τον υπολογισμό της απόστασής τους, χρησιμοποιείται η συνάρτηση `pairwise_distances` της βιβλιοθήκης `scikit-learn`, η οποία μπορεί να υπολογίσει μήτρες αποστάσεων με τη χρήση διάφορων μετρικών, συμπεριλαμβανομένης και της μετρικής Jaccard. Έχοντας υπολογίσει τη μήτρα αποστάσεων των χρηστών, ομαδοποιούμε τα δεδομένα με τη χρήση του αλγορίθμου `AgglomerativeClustering` της βιβλιοθήκης `scikit-learn`, ο οποίος μπορεί να τρέξει πάνω σε μήτρες αποστάσεων.

$$dist(u, v) = 1 - \frac{|\varphi(u) \cap \varphi(v)|}{|\varphi(u) \cup \varphi(v)|}$$

Εικόνα 3 - Τύπος απόστασης Jaccard

```
# Create a binary user-item matrix
# 1 if user rated the movie, 0 otherwise
user_item_matrix_binary = coo_matrix(
    (np.ones(filtered_df.shape[0]), (filtered_df['user_idx'],
    filtered_df['movie_idx'])),
    shape=(len(U_hat), len(I_hat)))
).toarray()

# Compute the Jaccard distance matrix between users
jaccard_distances = pairwise_distances(user_item_matrix_binary,
metric='jaccard')
```

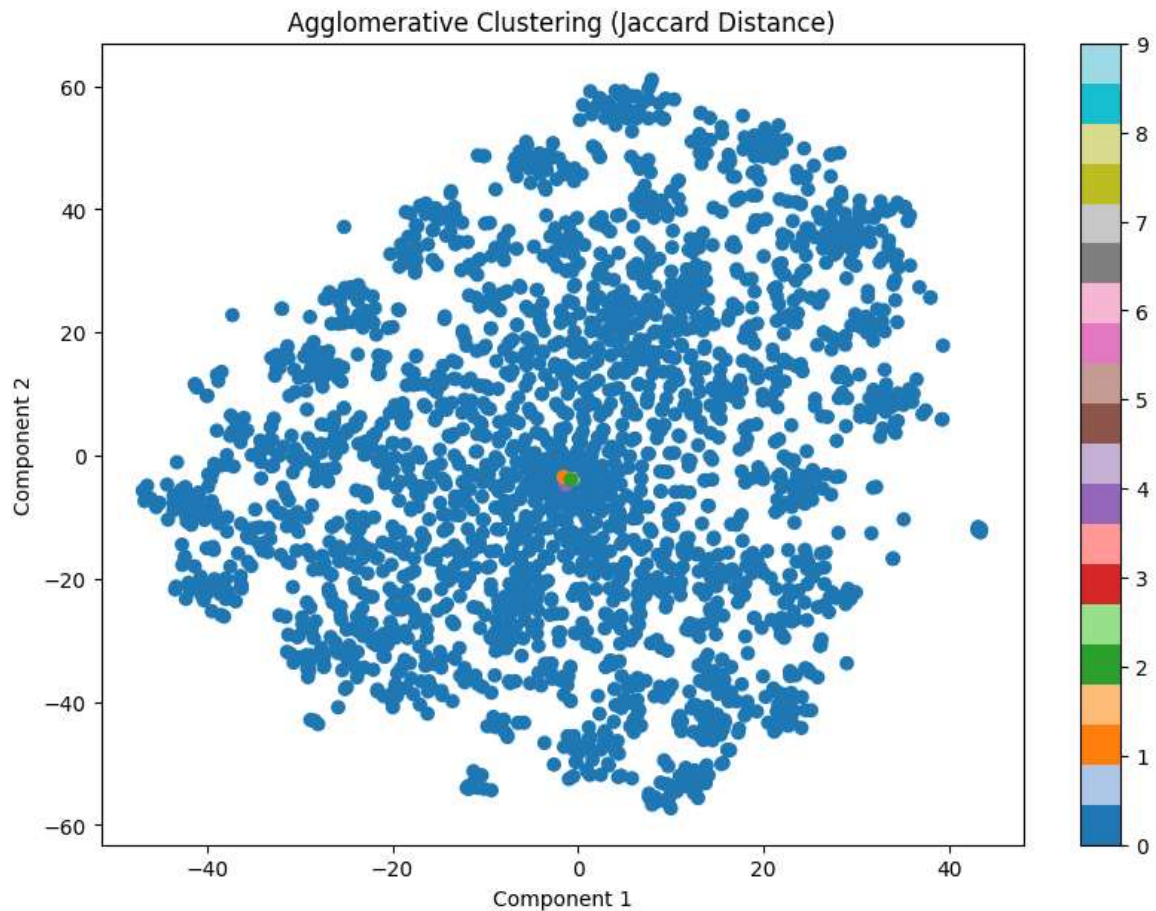


```
# Perform Agglomerative Clustering using the Jaccard distance matrix
clustering_model = AgglomerativeClustering(n_clusters=10,
metric='precomputed', linkage='average')
cluster_labels = clustering_model.fit_predict(jaccard_distances)
```

Για την γραφική αναπαράσταση των αποτελεσμάτων, μειώνουμε αρχικά τις διαστάσεις των δειγμάτων σε 50 με τη χρήση της μεθόδου SVD και έπειτα σε 2 με τη χρήση t-SNE.

```
# Visualize clusters using dimensionality reduction
svd = TruncatedSVD(n_components=50, random_state=42)
svd_data = svd.fit_transform(user_item_matrix_binary)
reducer = TSNE(n_components=2, random_state=42)
reduced_data = reducer.fit_transform(svd_data)

plt.figure(figsize=(10, 7))
scatter = plt.scatter(reduced_data[:, 0], reduced_data[:, 1],
c=cluster_labels, cmap='tab20')
plt.title('Agglomerative Clustering (Jaccard Distance)')
plt.xlabel('Component 1')
plt.ylabel('Component 2')
plt.colorbar(scatter)
plt.show()
```

Η μετρική Jaccard, αντιπροσωπεύει την απόσταση μεταξύ δύο χρηστών βάσει του πόσες κοινές ταινίες έχουν βαθμολογήσει. Όσο πιο πολλές κοινές ταινίες έχουν βαθμολογήσει, το πιο παρόμοιοι θεωρούνται οι χρήστες. Οι αποστάσεις Jaccard παίρνουν τιμές μεταξύ του 0 και του 1, με το μηδέν να σημαίνει ότι οι χρήστες έχουν βαθμολογήσει ακριβώς τις ίδιες ταινίες και 1 ότι οι χρήστες έχουν βαθμολογήσει εντελώς διαφορετικές ταινίες. Αυτή η μετρική λαμβάνει υπ, όψη ποιες ταινίες έχουν βαθμολογήσει οι χρήστες, αλλά όχι τις τιμές των κριτικών. Αυτό είναι το κύριο μειονέκτημα της μετρικής, σε σχέση με την ευκλείδεια και την συνημιτονική απόσταση. Η μετρική Jaccard, ομαδοποιεί τους χρήστες βάσει μόνο του ποιες ταινίες έχουν βαθμολογήσει, παραλείποντας σημαντικές πληροφορίες για τα δεδομένα, όπως την τιμή και το μοτίβο των κριτικών των χρηστών, παρέχοντας έτσι λιγότερο χρήσιμες και έμπιστες μετρικές για τις προτιμήσεις των χρηστών. Το πρόβλημα γίνεται πιο προφανές όταν εξετάζουμε ένα μικρό κομμάτι του αρχικού dataset, όπως στο παράδειγμα της υποενότητας. Στο παράδειγμα, ομαδοποιούμε μόνο τους χρήστες που έχουν αφήσει από 30 μέχρι 35 κριτικές. Παρ' όλο που σε αυτό το κομμάτι του dataset υπάρχει αρκετή ποικιλομορφία στα μεγέθη των δεδομένων, τα πλήθη των κριτικών των χρηστών περιορίζονται σε πολύ μικρή εμβέλεια, και γι αυτό οι περισσότεροι χρήστες ομαδοποιούνται στην ίδια συστάδα. Συνοπτικά, η μετρική Jaccard αγνοεί πολλές πληροφορίες για τα δεδομένα μας και δεν μπορεί να δημιουργήσει χρήσιμες συστάδες.



4.2 Πρόβλεψη προτιμήσεων χρηστών με τη χρήση των k-πλησιέστερων γειτόνων τους

Σε αυτή την υπο-ενότητα, θα χρησιμοποιήσουμε τη μετρική Jaccard για να βρούμε τους k πλησιέστερους γείτονες των χρηστών, τους οποίους θα χρησιμοποιήσουμε για να εκπαιδεύσουμε πολυστρωματικά νευρωνικά δίκτυα, ώστε να προβλέπουν τις προτιμήσεις των χρηστών, βάσει των ποιων ταινιών έχουν βαθμολογήσει. Θα πραγματοποιήσουμε τις προβλέψεις για τους 5 κοντινότερους γείτονες. Ορίζουμε τους γείτονες στη μεταβλητή k. Στη μεταβλητή n_clusters κρατάμε το πλήθος των συστάδων που δημιουργήθηκαν από την ομαδοποίηση με την απόσταση Jaccard. Θα πρέπει να εκπαιδευτούν διαφορετικά μοντέλα για κάθε συστάδα. Στο dictionary models θα αποθηκεύονται τα εκπαιδευμένα μοντέλα. Στα dictionaries training_mae και testing_mae αποθηκεύονται τα μέσα απόλυτα σφάλματα των μοντέλων των διαφορετικών συστάδων, για την εκπαίδευση και το testing τους αντίστοιχα. Η εκπαίδευση θα τρέχει μόνο για συστάδες που περιέχουν k + 1 στοιχεία, ώστε να έχει νόημα η εύρεση των k πλησιέστερων γειτόνων ενός στοιχείου. Οι συστάδες για τις οποίες πραγματοποιείται όντως η εκπαίδευση αποθηκεύονται στον πίνακα clusters_processed.

```
# Assuming you have already computed the Jaccard distances and obtained the
clusters
# Let's say cluster_labels is the array containing the cluster assignment for
each user

k = 5 # Number of nearest neighbors
n_clusters = len(np.unique(cluster_labels)) # Number of clusters

# Initialize dictionaries to store the trained models and MAE (Mean Absolute
Error) metrics for each cluster
models = {}
training_mae = {}
testing_mae = {}

# Track which clusters have been successfully processed
clusters_processed = []
```

Για κάθε cluster, μαζεύουμε τους δείκτες στο dataset των χρηστών που ανήκουν στο cluster. Ελέγχουμε αν το cluster περιέχει αρκετούς χρήστες ώστε να έχει νόημα η εύρεση των k πλησιέστερων γειτόνων των στοιχείων. Εάν το cluster έχει αρκετούς χρήστες, συνεχίζουμε την επεξεργασία του. Ανακτούμε από το dataset μας τα διανύσματα προτιμήσεων των χρηστών του cluster και υπολογίζουμε τις αποστάσεις Jaccard μεταξύ τους.

```
# Iterate over each cluster
for cluster in range(n_clusters):
    # Identify the indices of users belonging to the current cluster
    cluster_users = np.where(cluster_labels == cluster)[0]
```



```
# Skip this cluster if the number of users is too small to select k
nearest neighbors
if cluster_users.size < k + 1:
    continue

# Extract the user-item interaction matrix for users in this cluster
cluster_user_item_matrix = user_item_matrix[cluster_users, :].toarray()

# Calculate the Jaccard distance matrix for all users within this cluster
cluster_jaccard_distances = pairwise_distances(cluster_user_item_matrix,
metric='jaccard')
```

Συνεχίζουμε με την προετοιμασία των δεδομένων εκπαίδευσης και αξιολόγησης των μοντέλου του cluster. Στον πίνακα X θα αποθηκεύουμε τα χαρακτηριστικά εισόδου για κάθε δείγμα, τα οποία θα είναι οι προτιμήσεις των k πλησιέστερων γειτόνων ενός χρήστη, και στον πίνακα y θα αποθηκεύουμε τις προτιμήσεις του χρήστη ως τα αποτελέσματα που θέλουμε να λαμβάνουμε από τις τιμές του X .

```
# Initialize lists to store the input features (X) and target outputs (y)
for training the neural network
X = []
y = []
```

Για κάθε χρήστη του cluster, βρίσκουμε τους k κοντινότερους γείτονές του, ταξινομώντας τις αποστάσεις του από τους άλλους χρήστες και κρατώντας τα στοιχεία από τον δείκτη 1 έως το $k+1$. Ξεκινάμε από το δείκτη 1, καθώς στο δείκτη 0 θα βρίσκεται η απόσταση του χρήστη από τον εαυτό του, η οποία θα είναι πάντα 0. Μετατρέπουμε το δισδιάστατο πίνακα με τα διανύσματα προτιμήσεων των γειτόνων του χρήστη σε μονοδιάστατο και τον αποθηκεύουμε στον πίνακα με διανύσματα εισόδου. Τις βαθμολογίες του χρήστη, δηλαδή το διάνυσμα που περιμένουμε ως αποτέλεσμα του διανύσματος εισόδου, τις αποθηκεύουμε στον πίνακα y .

```
# Iterate over each user in the cluster
for i, user in enumerate(cluster_users):
    # Find the indices of the k-nearest neighbors for the current user,
    excluding the user itself
    nearest_neighbors_idx =
np.argsort(cluster_jaccard_distances[i])[1:k+1]

    # Gather the ratings of the k-nearest neighbors
    neighbor_ratings = cluster_user_item_matrix[nearest_neighbors_idx, :]
```




```
# Flatten the neighbor ratings into a single vector and add it to the
input list
X.append(neighbor_ratings.flatten())

# The target output is the user's own rating vector
y.append(cluster_user_item_matrix[i])

# Convert the input and output lists to numpy arrays
X = np.array(X)
y = np.array(y)
```

Χωρίζουμε τα δεδομένα μας σε δεδομένα εκπαίδευσης και δεδομένα αξιολόγησης σε ποσά 80/20. Έπειτα, κατασκευάζουμε το νευρωνικό δίκτυο για το cluster. Θα κατασκευάσουμε ένα πολυστρωματικό νευρωνικό δίκτυο με δύο κρυφά επίπεδα. Αρχικοποιούμε το δίκτυο με τη μέθοδο Sequential, που σημαίνει ότι το output ενός επιπέδου του δικτύου θα είναι το input του επόμενου. Προσθέτουμε δύο κρυφά στρώματα, ένα με 128 νευρώνες και ένα με 64, με την ReLU ως συνάρτησης ενεργοποίησης. Τέλος, προσθέτουμε ένα επίπεδο εξόδου με features για όλες τις πιθανές κριτικές που μπορεί να αφήσει ένας χρήστης. Εκπαιδεύουμε το μοντέλο και το αποθηκεύουμε στον πίνακα models.



```
# Split the data into training and testing sets (80% training, 20%
testing)
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2,
random_state=42)

# Define the architecture of the neural network model
model = Sequential()
# First hidden layer with 128 neurons
model.add(Dense(128, input_dim=X_train.shape[1], activation='relu'))
# Second hidden layer with 64 neurons
model.add(Dense(64, activation='relu'))
# Output layer with linear activation
model.add(Dense(y_train.shape[1], activation='linear'))

# Compile the model using Adam optimizer and mean squared error loss
function
model.compile(optimizer='adam', loss='mean_squared_error')

# Train the model on the training data
model.fit(X_train, y_train, epochs=10, batch_size=32,
validation_data=(X_test, y_test))

# Store the trained model in the dictionary with the cluster number as
the key
models[cluster] = model
```

Δοκιμάζουμε την ακρίβεια του μοντέλου. Κάνουμε προβλέψεις για τα δεδομένα με τα οποία εκπαιδεύτηκε το μοντέλο και για τα δεδομένα αξιολόγησης και υπολογίζουμε το Mean Absolute Error των προβλέψεων σε σχέση με τα αναμενόμενα αποτελέσματα. Κρατάμε το label του cluster στον πίνακα `clusters_processed`, καθώς έχει κατασκευαστεί μοντέλο για το cluster αυτό.

```
# Make predictions on the training set
y_train_pred = model.predict(X_train)

# Make predictions on the testing set
y_test_pred = model.predict(X_test)

# Calculate the Mean Absolute Error (MAE) on the training set and store
it for the current cluster
training_mae[cluster] = mean_absolute_error(y_train, y_train_pred)
```



```
# Calculate the Mean Absolute Error (MAE) on the testing set and store it
for the current cluster
testing_mae[cluster] = mean_absolute_error(y_test, y_test_pred)

# Keep track of which clusters have been processed
clusters_processed.append(cluster)
```

Εκτυπώνουμε τα MAE εκπαίδευσης και αξιολόγησης για τα μοντέλα που κατασκευάστηκαν.

```
# Print the MAE results for each processed cluster
for cluster in clusters_processed:
    print(f"Cluster {cluster + 1}:") # Cluster numbers are 1-based for
display
    print(f"  Training MAE: {training_mae[cluster]}") # Print the training
MAE
    print(f"  Testing MAE: {testing_mae[cluster]}") # Print the testing MAE
```

Cluster 1:

Training MAE: 0.019458982783917955
Testing MAE: 0.016903112514026958

Τα σφάλματα για τα δεδομένα εκπαίδευσης είναι αρκετά χαμηλά, που σημαίνει ότι μοντέλο προβλέπει τις προτιμήσεις των χρηστών με μεγάλη επιτυχία. Το σφάλμα για τα δεδομένα αξιολόγησης είναι ελαφρώς μικρότερο, που υποδεικνύει ότι ίσως τα δεδομένα αξιολόγησης είναι πιο προβλέψιμα από τα δεδομένα εκπαίδευσης.



5 Βιβλιογραφία

- [Pik12] Πικράκης Α., Κουτρούμπας Κ. και Γιαννακόπουλος, Θ., 2012. «Αναγνώριση Προτύπων». Broken Hill.
- [np] Numpy Documentation: <https://numpy.org/doc/stable/index.html>
- [sklearn] scikit-learn Documentation: <https://scikit-learn.org/stable/>
- [tf] TensorFlow Documentation: <https://www.tensorflow.org/>
- [mpl] matplotlib Documentation: <https://matplotlib.org/>
- [scipy] SciPy Documentation: <https://scipy.org/>